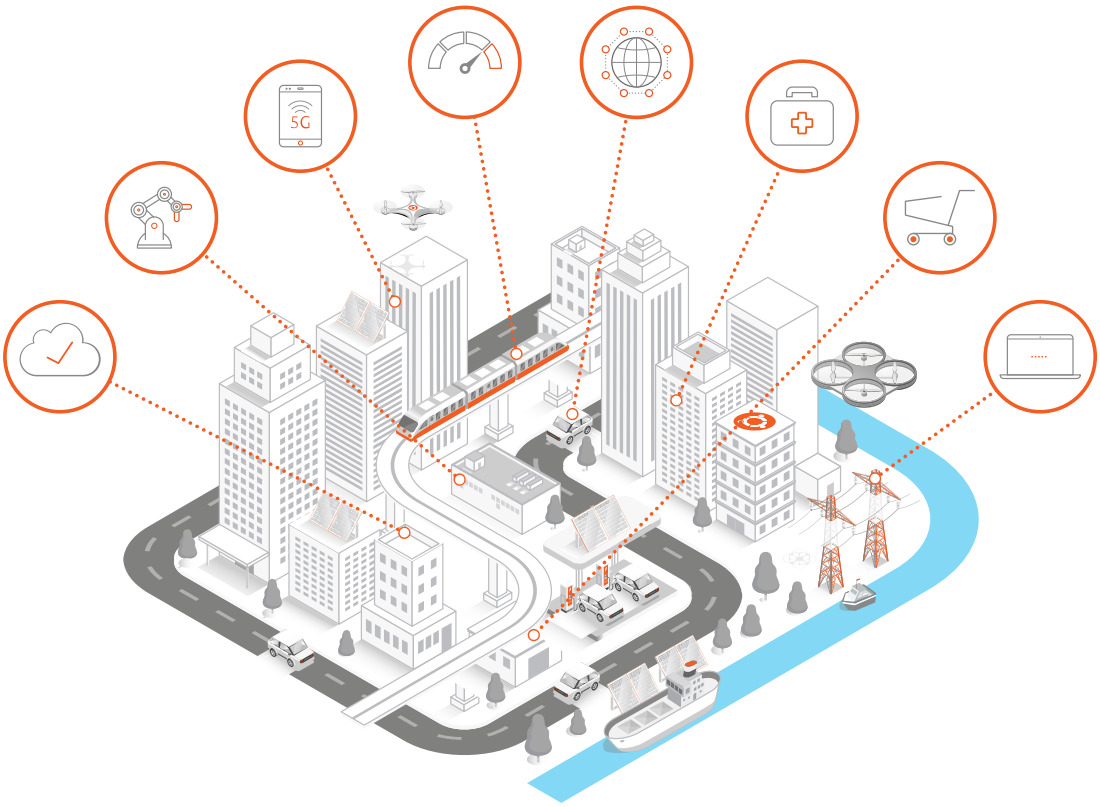
Canonical

# How Ubuntu Core transforms OTA updates for IoT

Over-the-air software updates delivered in a secure, reliable way

January 2023



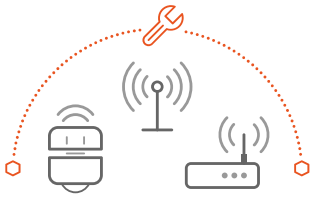

## Executive summary

Delivering consistent, reliable and robust software updates to any platform is a huge challenge. Delivering consistent, reliable and robust software updates to low-powered, inaccessible, and often remotely administered embedded IoT devices, anywhere in the world, is intricately harder.

Ubuntu Core and snaps elegantly solve these problems, at any scale with any release cadence. With Ubuntu Core, updates are delivered to a single device, a specific subset of devices, or a deployment of tens of thousands of devices, with an emphasis on maintaining a system's integrity.

This paper outlines the challenges pertaining to software updates on embedded devices, and how Canonical has harnessed its unparalleled expertise and experience to solve them. It explores advanced features of Ubuntu Core, the operating system designed for IoT, and the Snap Store that deliver OTA updates seamlessly.

# The challenges of OTA updates



Whether it's smart cities, wearable technology, robotics, autonomous vehicles, or any other new and emerging IoT sector, network connectivity is central to IoT's advancement. A recent forecast by *IDC*, for instance, suggests such devices will number over 41.6 billion by 2025, generating almost 80 zettabytes of data. As the number of data and devices grow, so does the revenue potential of the ioT sector. According to *McKinsey & Company's recent report*, it is estimated that by 2030, IoT could enable $5.5 trillion to $12.6 trillion in value globally. This includes the value captured by consumers and customers of IoT products and services. With this extensive connectivity comes the huge responsibility of keeping devices updated, patched, and secure. This responsibility falls predominantly on the update and delivery mechanisms IoT vendors choose.

For decades, updates and their management systems have replaced existing files and binaries on the system, often in long interdependent chains. If anything goes wrong, such as a software bug, power outage or filesystem issue, the system becomes impaired. This often requires costly, expert manual intervention to recover.

An even trickier problem occurs when the update process works but the update itself is faulty and needs to be reverted. This problem is not trivial to solve on traditional systems, because downloading an old version from a package repository and installing it over a broken system requires specific scripts from a package maintainer.

Historically, the options to handle failed updates or software regressions were to send engineers out to affected sites, recall the devices or remotely update each device individually. Each option is resource-heavy, increases OPEX costs and is not scalable as deployment sizes grow. An effective update solution, therefore, needs to include all of the following:

- **Automated updates to any device** - the update service needs to be a reliable and automatic process, catering for a predictable update cadence and an optional fine level of control over when and how updates are delivered.
- **Mechanisms for update recovery** - the inaccessible nature of many embedded device installations makes it imperative that updates are solid and capable of withstanding faulty, blocked, partial and interrupted upgrades.
- **Critical update provision** - in specific circumstances when an ad-hoc or critical update needs to be made, the update system needs to give these priority and incorporate the update back into the regular update service.
- **Redundancy** - in situations that can't be easily modelled or predicted, an update system needs to have enough redundancy to handle roll-backs, network-free bootstrapping and autonomous re-provisioning.

These are intricately complex challenges that face all embedded IoT systems. This is the reason why updates have previously required serious long-term commitment from IoT vendors. Such commitments may deteriorate into infrequent, low-quality update payloads, often leading to compromised functionality, frustrated users, distrust and poor adoption.

The update problem can only be solved holistically; from early development through to production and deployment. The solution needs to focus on guaranteeing the integrity of operating system images, and how they remain secure, reproducible, and reliable, no matter which phase of development your project is in.

## How does Ubuntu Core solve OTA challenges?

Ubuntu Core

Ubuntu Core is the embedded variant of Ubuntu, tailor-made for IoT and embedded devices. It's an ultra-secure, resilient, and application-centric operating system, backed by a growing ecosystem of silicon and ODM partners. Ubuntu Core is made up of snaps. Snaps are a secure, dependency-free packaging system, able to work without modification across all Linux distributions. Snaps can be fully containerised, ensuring there is always a clean separation between the base system and whatever applications need to be installed. This also enables isolation between every application and its data (including even application version data). Snaps are published and distributed to the global Snap Store, where anyone can browse and find the software they need. Specifically for IoT, we have designed a dedicated Snap Store, or IoT App Store, allowing software to be uploaded privately and distributed to fleets of devices with any desired configuration.

Ubuntu Core harnesses sophisticated update technology, used by devices daily around the world. Updates are transactional, which means that if an update is not successful, it is not installed. If it's not installed, the system reverts to its last stable state and leaves no trace of any failure other than log details. This means the system remains fully operational and in a continuously well-defined state during both application and system updates. The system can also recover or revert to previous states if necessary, even if a system fails to boot. Unlike alternative or more traditional package managers, a failed update never leaves the system in an unpredictable state. This solution was designed specifically to solve the complex problems associated with updating critical software. It makes updates autonomous, reliable and secure, whilst still offering a broad and carefully selected level of control over an update's deployment.

Ubuntu Core and snaps revolutionise update methodologies with three unique approaches to the cornerstones of IoT deployment:

- **Security**
- **Update mechanics and recovery**
- **System maintenance**

Let's cover these aspects in more detail.

# Security

Security is a vital consideration for devices, especially where host applications run autonomously in trusted, inaccessible environments. Security and vulnerability mitigation is often neglected, and seldom built into the fabric of a platform. In fact, it's too often squeezed into a software stack after the customer-facing functionality has been designed, developed and iterated over.

According to a recent *Open Source Security and Risk Analysis Report*, 100% of codebases in the IoT sector contained open source, and an astounding 92% of the audited code in this sector was open source. However, troublingly, 64% of the IoT codebases also contained vulnerabilities. Security often only becomes important (and costly) when vulnerabilities are discovered. What's worse is that many IoT devices are built on custom-built software stacks, unique to each device. It can be easier to abandon these deployments, leaving them unprotected, than take on the challenge of updating them effectively.

To prevent this, Ubuntu Core and snaps are built on immutable foundations. These harness the same strengths of both the Linux kernel and Ubuntu that make them so successful for millions of embedded, desktop and cloud users. The base system of Ubuntu Core includes only what's necessary for the operating system to run. This includes little more than the kernel, an init process, a few essential tools, and snapd (the daemon that manages snaps). The snap daemon uses a watertight combination of open source technologies like *cgroups, Seccomp* and *AppArmor* to ensure a snap is always isolated from the system and from every other snap, whilst still allowing granular control over which resources can be accessed and shared.

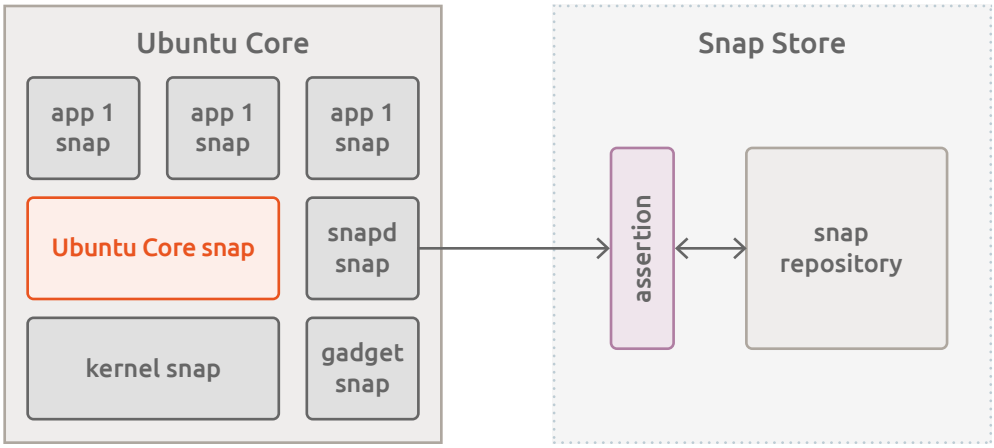



*Diagram showing Ubuntu Core, consisting of key components of the operating system packaged as snaps, communicating with the Snap Store to receive OTA snap updates.*

Without explicit approval for the privileges assigned within an assertion, snaps have very limited access to the system. Snaps cannot make changes to another user, access hardware, add users, change security policy, modify the system, modify kernel runtime variables, and many other functions necessary to subvert a system. In addition, snaps themselves, when not pre-installed, are always delivered and updated from a trusted source attested by a digitally signed assertion.

To reduce the overhead for vendors packaging their own applications, the Snapcraft build tool is available. With this tool, most apps require little more than the creation of a configuration file to describe application requirements. Snapcraft integrates these requirements within its build framework to ensure the resulting snap is secure, compliant and efficient. This process massively reduces both the total cost of ownership and the time it takes to get a product to release. The build processes and the base operating system are open source, highly available and have been widely used for many years.

One example of a business taking advantage of these unique features is Screenly, a world-leading digital signage platform provider, powering over 10,000 screens around the world. Screenly uses Ubuntu Core with a dedicated IoT App Store to deliver seamless over-the-air updates to devices in the field.

An effective digital sign must be stable, easy to manage and secure. To keep deployed devices up-to-date with the latest security fixes and features, Screenly originally relied on home-grown infrastructure to deliver over-the-air (OTA) patches to devices in the field. However, this framework was expensive and time consuming to maintain. As the fleet scaled up, devices were occasionally being left nonfunctional in partially updated states. This forced the company's development team to spend significant time writing recovery code. In addition to security implications, these difficulties also impacted screen stability. Without an effective way to keep the software versions of deployed devices in line, compatibility issues were arising that resulted in screen downtime and troubleshooting headaches.

Rather than continuing to pump time and resources into its in-house OTA infrastructure, Screenly decided to look for an existing solution that would enable it to leverage the established expertise of a trusted third-party partner. After a comprehensive market evaluation, Ubuntu Core stood out as an ideal fit.

"Ubuntu Core ticked every box," notes Viktor Petersson, CEO at Screenly. "The containerised approach with snaps is a great way to maximise security, and the IoT App Store gives us the prebuilt infrastructure to seamlessly deliver OTA updates to our entire fleet."

Today, Screenly relies on Canonical solutions to rapidly deploy software updates to thousands of digital signage players in the field, with none of the friction and at a fraction of the cost associated with its legacy infrastructure. From the customer perspective, screen downtime and security vulnerabilities are practically unheard of. Additionally, Screenly's customers no longer have to worry about software patches or take any action to update devices themselves. Screenly takes care of everything through the IoT App Store, ensuring a smooth, hands free experience for its users. In fact, "Automatic Updates" now features prominently on Screenly's homepage as one of its key value propositions. Read more about Screenly's story.

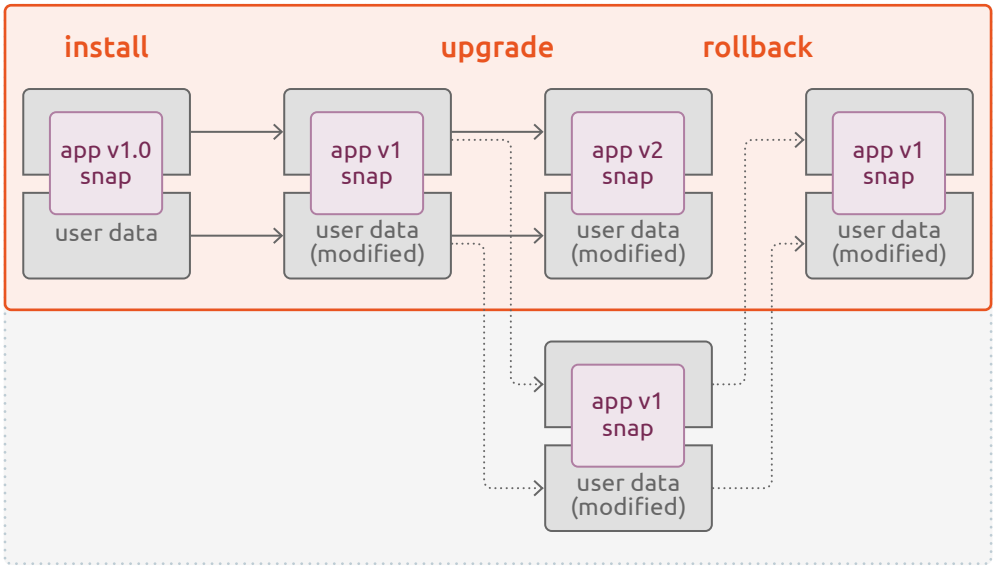
# Update mechanics and recovery

Besides security challenges, updates can be easy, reliable and robust when they happen autonomously. After a new snap has been published to Canonical's global Snap Store, or a private IoT App Store (a dedicated Snap Store) controlled by the IoT vendor, it's then pushed to devices automatically via their configurable refresh cadence. Updates are transferred as deltas, a form of differential data compression that only transmits the changes between the currently installed snap and the new one. Deltas save considerable bandwidth, which has a huge impact when multiplied across a deployment of thousands of IoT devices that could need multiple critical updates over a short period of time.

During an update, a snap's data is always safe. Each snap has its own snap-specific, segmented area for data storage which is separate from the general writeable operating system files and the state of the snap daemon itself. When a snap is updated, whatever its level in the stack, its data is copied and made accessible to the new snap. If the update transaction fails for any reason, the snap can be rolled-back to its previous state, including its data.

Extra care is taken with the kernel and the bootable base snap. When a new kernel or bootable base is installed, Ubuntu Core enables a special `try-boot` mode. If the subsequent reboot fails, the system automatically reverts back to the previous working kernel or base. Similarly, each revision of a snap has its own data directory, which means that when something is broken and needs to be rolled back, the data of the previous revision is also restored. Any undetected and incompatible change in the new version does not break the rollback.

### Transactional updates in Ubuntu Core




*If any updates fail, the system is protected. The application falls back to its last stable state, without risk of breakage.*

By default, Ubuntu Core locally archives the two previous versions of a snap in an inactive state. If there are problems with the new version of a snap, a simple `snap revert [problematic-snap]` will get the system back to the previous working version. And because each archived snap includes its own data, a rollback will also restore pre-update data. This means that data can be recovered from an archived previous revision, unless the snap publisher opted out of this feature.

Ubuntu Core offers a secure recovery system that can be activated manually when booting, or remotely via API call. In recovery mode, Ubuntu Core presents an interface for users to enter recovery options in the case of system failure. The operating system then proceeds to fetch the configuration files and snaps associated with a given configuration profile, allowing the system to revert to a stable state without damage. Multiple snapshots for the same device can be backed up in the recovery system. These snapshots reflect both configuration settings and the collection of snaps installed on the system. Device operators can create, name and save such snapshots in the recovery system. This enables a swift return to the desired system state when needed. Recovery can happen manually through a dedicated UI or remotely via an API call.

With Ubuntu Core, both the operating system and applications are protected, without risk of breakage.

## System maintenance

For a system to function correctly, it must be maintained. Maintenance often falls under two categorisations: preventative and corrective. Preventative maintenance prevents the system falling into an unmanageable state. Corrective maintenance is done to fix a system that encounters issues during operation. This shows that maintenance isn't simply the ongoing process of keeping everything up-to-date and as clean as possible. There are multiple aspects to consider.

In general, maintenance is a process that has to start with good design. Without good design, any deployment will have a greater propensity to spiral into an unknown state, with files littered across the filesystem, temporary files un-checked, orphaned configuration files and applications unmonitored. The longer the system is left in this state, the increasingly harder it is to revert. This all adds maintenance and deployment complexity. This also increases the burden on maintainers, until it typically passes a point where maintenance becomes untenable. Often, the system is then insecure and will remain insecure until, ultimately, it's reset into a known good state. With good design to start, these issues can be avoided altogether.

Good design is at the heart of Ubuntu Core, a system designed to be effortlessly maintainable for the following reasons:

- Ubuntu Core's filesystem is read-only
- Configuration and application data is confined to each snap
- Data is always in a known state
- Updates are signed and atomic
- Applications are confined and cannot modify each other without permission

This last point wouldn't be possible without snaps, as they are a single file mounted directly into the host filesystem. Like the core filesystem, they're presented to the system as read-only and are granted access to whatever resources they need through a set of explicit permissions, known as interfaces. Interfaces are implemented using well-tested Linux kernel confinement features.

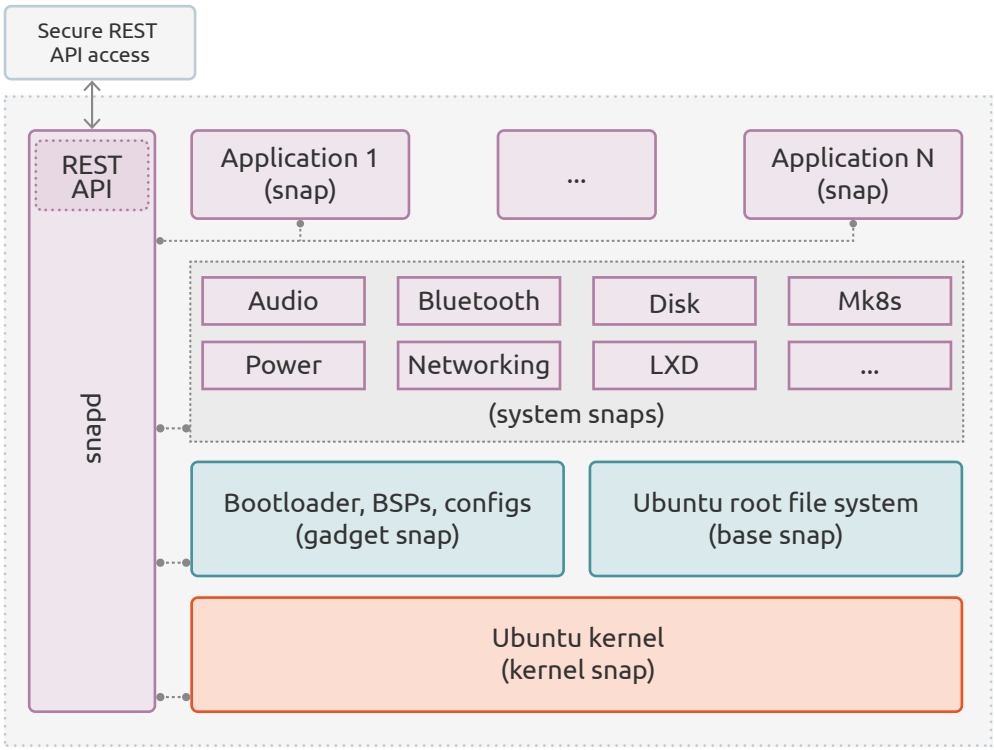



*Diagram showing the components of Ubuntu Core, all isolated individual snaps, communicating with snapd.*

Snaps themselves are entirely self-contained, even encapsulating their own filesystem. This means that they include everything an application needs to run that isn't provided by Ubuntu Core. Ubuntu Core itself is built from snaps; there's a kernel snap that includes the Linux kernel and hardware-specific drivers, the gadget snap that describes the specifics of any particular model of device, and a snap for the snapd daemon itself. This orchestrates the snap installation and update procedure, as well as how the assertions that define the permissions for a snap are translated into local system privileges.

Snaps also allow for multiple versions of the same snap to be available on the system at the same time, with no adverse effects. This mechanism, known as parallel installation, enables independent instances of the same snap to be installed and run from within their own namespaces. This enables vendors to concurrently support both an old version and a new version of an application, library or development platform until such a time that the old version can be deprecated. As with any other snap, each instance is completely isolated from all other instances. This includes a snap's name, configuration, interface connections, data locations, services, applications and aliases.

In addition, thanks to another feature called epochs, if an application's configuration or data format changes considerably, such as with a major version release, users of the older incompatible version won't automatically be updated when it becomes available. Instead, the vendor has complete control over which users receive an update and when, allowing older users to update only when a tested migration policy becomes available. What's more, this process stays in place for any device that finds itself initialised with the original older version, which will be updated automatically through the same migration policy.

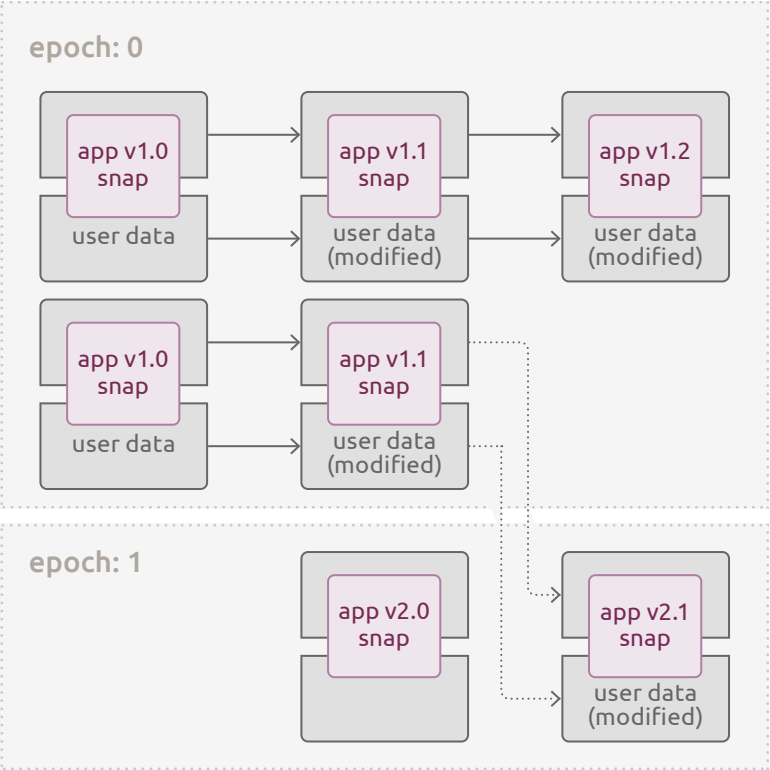



*Diagram showing parallel installs and epochs.*

All of this is wrapped into a streamlined CI/CD build and publishing process that will automatically create and publish snaps after any committed changes. Snap channels, tracks and branches can then be used to set stability expectations for a release. For example, the edge channel can be used to track cutting edge features, while the stable channel can be reserved only for releases that include long term support from the publisher.

Using these well-designed features of Ubuntu Core and snaps results in minimising the system maintenance required, allowing more focus and time spent on the advancement of your IoT project.

# Secure IoT updates for the future

Delivering consistent, reliable and robust software updates to low-powered, inaccessible, and often remotely administered embedded systems is a difficult challenge.

With Ubuntu Core and snaps, updates are a solved problem. It's a solution that rewards the success of growing IoT deployments - not with increasing complexity and uncertainty, but with simple confidence, regardless of their deployment scale or update frequency.

These advantages are not just in the way updates are delivered, but also in the way they can be planned, scheduled and published predictability. They can reduce the total cost of ownership of a device, and help a vendor take their product from design to deployment in far less time than having to engineer the build tools, the update mechanism and the delivery infrastructure.

If you are looking for ways to advance your next IoT project, please get in touch or chat with us in the forum.

## Resources

Learn more about Ubuntu Core and the dedicated Snap Store for IoT

- Top 5 IoT challenges and how to solve them
- A practical guide to IoT lifecycle management
- How to reduce your IoT costs with Ubuntu Core
- Embedded Linux: make or buy?
- 4 ways to accelerate time to market for IoT devices

© Canonical Limited 2023. Ubuntu, Kubuntu, Canonical and their associated logos are the registered trademarks of Canonical Ltd. All other trademarks are the properties of their respective owners. Any information referred to in this document may change without notice and Canonical will not be held responsible for any such changes.

Canonical Limited, Registered in Isle of Man, Company number 110334C, Registered Office: 2nd Floor, Clarendon House, Victoria Street, Douglas IM1 2LN, Isle of Man, VAT Registration: GB 003 2322 47

Ubuntu